

Data structures and Algorithms

Exercise 2: E-commerce Platform Search Function

Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

1. Understand Asymptotic Notation:

- Explain Big O notation and how it helps in analyzing algorithms.
- Describe the best, average, and worst-case scenarios for search operations.

2. Setup:

- Create a class **Product** with attributes for searching, such as **productId**, **productName**, and **category**.

3. Implementation:

- Implement linear search and binary search algorithms.
- Store products in an array for linear search and a sorted array for binary search.

4. Analysis:

- Compare the time complexity of linear and binary search algorithms.
- Discuss which algorithm is more suitable for your platform and why.

Understanding Asymptotic Notation:

Big O Notation

Big O notation is a mathematical way to describe how the runtime or space requirements of an algorithm grow as the input size grows. It provides an upper bound on the time, giving the "worst-case" scenario, which helps developers ensure their application will scale without crashing under heavy loads.

Search Scenarios

Best-case: The item is found immediately (e.g., at the first index). Both Linear and Binary search are $O(1)$ in the best case.

Average-case: The item is found somewhere in the middle. Linear search is $O(n/2)$ which simplifies to $O(n)$. Binary search is $O(\log n)$.

Worst-case: The item is at the very end or doesn't exist. Linear search is $O(n)$. Binary search is $O(\log n)$.

CODE:

Main.java:



```
J Main.java > ...
1 import java.util.Arrays;
2
3 class Product implements Comparable<Product> {
4     private int productId;
5     private String productName;
6     private String category;
7
8     public Product(int productId, String productName, String category) {
9         this.productId = productId;
10        this.productName = productName;
11        this.category = category;
12    }
13
14    public int getProductId() {
15        return productId;
16    }
17
18    public String getProductName() {
19        return productName;
20    }
21
22    @Override
23    public int compareTo(Product other) {
24        return Integer.compare(this.productId, other.productId);
25    }
26
27    @Override
28    public String toString() {
29        return "Product{" + "id=" + productId + ", name=" + productName + '\'' + '\'';
30    }
31 }
32
```

```

33 public class Main {
34
35     public static Product linearSearch(Product[] products, int targetId) {
36         for (Product product : products) {
37             if (product.getProductId() == targetId) {
38                 return product;
39             }
40         }
41         return null;
42     }
43
44     public static Product binarySearch(Product[] products, int targetId) {
45         int left = 0;
46         int right = products.length - 1;
47
48         while (left <= right) {
49             int mid = left + (right - left) / 2;
50
51             if (products[mid].getProductId() == targetId) {
52                 return products[mid];
53             }
54             if (products[mid].getProductId() < targetId) {
55                 left = mid + 1;
56             } else {
57                 right = mid - 1;
58             }
59         }
60         return null;
61     }
62 }

```

Run | Debug

```

Run | Debug
63 public static void main(String[] args) {
64     Product[] products = {
65         new Product(productId: 105, productName: "Laptop", category: "Electronics"),
66         new Product(productId: 101, productName: "Mouse", category: "Accessories"),
67         new Product(productId: 108, productName: "Keyboard", category: "Accessories"),
68         new Product(productId: 102, productName: "Monitor", category: "Electronics"),
69         new Product(productId: 110, productName: "Desk", category: "Furniture")
70     };
71
72     System.out.println(x: "--- Linear Search ---");
73     Product foundLinear = linearSearch(products, targetId: 108);
74     System.out.println("Found (108): " + (foundLinear != null ? foundLinear : "Not found"));
75
76     System.out.println(x: "\n--- Binary Search ---");
77     Arrays.sort(products);
78     Product foundBinary = binarySearch(products, targetId: 102);
79     System.out.println("Found (102): " + (foundBinary != null ? foundBinary : "Not found"));
80 }
81 }

```

OUTPUT:

```

--- Linear Search ---
Found (108): Product{id=108, name='Keyboard'}

--- Binary Search ---
Found (102): Product{id=102, name='Monitor'}

```

Analysis:

Time Complexity Comparison:

Linear Search: $O(n)$. It checks every element one by one. If we have 1 million products, it might take 1 million checks.

Binary Search: $O(\log n)$. It halves the search space every step. For 1 million products, it takes at most ~ 20 checks.

Conclusion: For an e-commerce platform where fast performance is required and the product catalog is likely very large, Binary Search is far more suitable. The only caveat is that the product array must be kept sorted, which is a manageable trade-off for the massive gain in search speed.